Semester Project Specification

Project Overview

This semester project is your opportunity to demonstrate the backend engineering skills covered in CPSC-449 by designing, implementing, documenting, deploying, and presenting a complete web back-end application. The project should combine multiple course concepts into one cohesive system that can be accessed and demonstrated through a working client, API interface, or deployed service.

The goal is to build a backend system that is technically complete, secure, maintainable, and substantial enough to demonstrate your understanding of API design, database management, authentication and authorization, real-time functionality, deployment, version control, and professional presentation.

Open Topic: Build Something You Care About!

This is an **open-topic and open-tool project**. You may build a backend application around a topic that is useful, interesting, creative, or personally meaningful to you.

Possible directions include:

- **Solve a real-world problem:** Build a service that improves an everyday workflow or helps users accomplish a useful task.
- **Build a product you wish existed:** Create the backend for an application or service that you would personally use.
- **Explore something you are passionate about:** Topics may include gaming, fitness, sports, music, food, travel, finance, education, productivity, entertainment, community activities, or other interests.
- **Extend an existing idea:** You may create an original implementation of a familiar type of application, as long as the design and implementation are your own work and satisfy all technical requirements.

Recommended Technology

The recommended backend stack is **Node.js** + **Express.js**, but other appropriate frameworks and languages may be used when justified by the project. Examples include Python with Flask or Django, ASP.NET, Ruby on Rails, Java/Spring, or another instructor-approved backend framework.

All projects must:

- Implement a functional backend service or API.
- Use a database for persistent application data.
- Use Git and GitHub for version control and project submission.
- Be deployed to a publicly accessible hosting platform by the final presentation.

Teamwork or Solo: Your Choice

You may work **individually** or as part of a **team of up to four students**. Individual projects are welcome, but they will be graded using the **same technical requirements** as group projects. Project requirements are not reduced based on team size.

For team projects:

- All members must make meaningful contributions to the codebase.
- All members must use Git/GitHub throughout development.
- Individual contribution may be evaluated using Git history, project check-ins, demonstrated code ownership, and oral questioning.
- Every team member should understand the overall project and be able to explain the portions they implemented.

Project Grading Summary

Category	**Points**
API Integration	15
Database Management	15
Backend Architecture, Authentication, and RBAC	30
Deployment	10
Version Control and Collaboration	10
Technical Requirements Total	**80**
Presentation	20
Project Total	**100**
Extra Credit	**Up to +10**

The completed project is worth **100 points total**: 80 points for technical implementation and 20 points for the final oral presentation. Students may earn up to **10 additional extra-credit points** by implementing approved advanced features that go meaningfully beyond the core requirements.

Technical Requirements (80 points total)

The sections below describe the minimum requirements and grading expectations for each category in detail.

- **API Integration (15 points)**
  - Integrate at least **3 APIs** into the application.
  - The required API set must include:
    - **1 Public API**: for example, OpenWeatherMap, NewsAPI, Google Maps, TMDb, Spotify, or another appropriate public service.
    - **1 Private API**: create a secure API that requires authentication, such as an API key, JWT, OAuth 2.0 token, or another protected access mechanism.
    - **1 Additional API**: may be public or private depending on project needs.
  - API data must be meaningfully incorporated into the project rather than included only for demonstration.
  - Students are responsible for understanding documentation, authentication requirements, rate limits, usage restrictions, and any costs associated with selected services.

  - API keys, tokens, secrets, and credentials must not be committed to a public GitHub repository.

- **Database Management (15 points)**
  - Use a database to store and manage application data.
  - Suggested options include MongoDB, PostgreSQL, MySQL, SQLite, Firebase, Supabase, or another appropriate database system.
  - Implement complete **CRUD operations**: Create, Read, Update, and Delete.
  - Database operations must be connected to actual application workflows and user actions.
  - Support **real-time or automatically synchronized updates** where appropriate to the project.
  - The application must include at least one real-time listener, subscription, WebSocket-driven update, polling workflow, or equivalent mechanism that allows relevant data changes to appear without requiring a full manual page refresh.
  - Examples include stock prices, game scores, bid values, statistics, cryptocurrency values, votes, polls, chat messages, collaborative tasks, or other live data.

- **Backend Architecture, Authentication, and RBAC (30 points)**
  - Clearly define the backend structure, including routes, controllers, services, models, middleware, or equivalent architectural components.
  - Organize the codebase so that major responsibilities are separated and maintainable.
  - Implement **User Authentication** using JWT, session-based authentication, OAuth, Firebase Authentication, Supabase Authentication, or another appropriate approach.
  - Protect user-specific functionality and restricted endpoints.
  - Implement **Role-Based Access Control (RBAC)** with at least **two user roles** that have meaningfully different permissions.
  - Example role combinations include `admin` vs. `user`, `manager` vs. `member`, `instructor` vs. `student`, or `seller` vs. `buyer`.
  - Include **protected routes** that enforce role-specific access.
  - Example: administrators may access user-management endpoints while regular users may access only their own account data.
  - Securely handle passwords, tokens, environment variables, secrets, and credentials.

- **Deployment (10 points)**
  - Deploy the backend application to a public hosting or cloud platform.
  - Appropriate platforms may include Render, Railway, Vercel, Netlify Functions, AWS, Azure, Google Cloud, Heroku, Fly.io, DigitalOcean, or another suitable service.
  - The deployed backend must be functional and accessible through a public URL during grading and presentation.
  - Required environment variables, database connections, API credentials, and production configuration must be set correctly in the deployment environment.

- **Version Control and Collaboration (10 points)**
  - Submit the project through a **GitHub repository**.

- Use Git throughout development with regular, meaningful commits.
- Avoid submitting the entire semester project as one large final commit.
- For team projects, every team member must make identifiable contributions to the repository.
- Use collaborative workflows such as branches, pull requests, code reviews, issues, or GitHub Projects when appropriate.
- Include a clear `README.md` containing:
  - Project title and description
  - Team members, if applicable
  - Major application features
  - Backend technology stack
  - APIs used
  - Database technology and data model overview
  - Authentication and authorization approach
  - Required environment variables and setup instructions
  - Local run instructions
  - Deployment URL
  - API endpoint summary or documentation link
  - Known limitations or unfinished features, if any

Project Check-Ins

Project check-ins are used to verify steady progress and reduce the risk of teams attempting to complete the entire project near the end of the semester. Specific dates and submission details will be posted in Canvas.

First Project Check-In – Proposal and Architecture

By the first check-in, students should be prepared to discuss:

- A clear project idea or problem statement
- Intended users and primary use cases
- Planned major features
- Proposed backend framework and database
- Planned public, private, and additional APIs
- Initial authentication and user-role design
- An established GitHub repository
- A preliminary architecture or route plan

The goal of this check-in is to confirm that the project scope is feasible and that the technical design can satisfy the course requirements.

Second Project Check-In – Core Backend Progress

By the second check-in, students should be prepared to demonstrate:

- A functioning backend server
- Initial routes and controllers/services
- Working database connection
- At least partial CRUD functionality
- Initial authentication functionality
- Meaningful Git history showing active development
- Progress on at least one external API integration

Final Technical Check-In – Deployment Readiness

By the final technical check-in, students should be prepared to demonstrate:

- Major backend routes and services functioning
- Complete or substantially complete CRUD functionality
- Working authentication
- Working RBAC with at least two user roles
- Progress toward all three required API integrations
- Real-time or automatically synchronized data functionality
- Deployment configuration or a functioning staging deployment
- Updated documentation and setup instructions

The completed project submitted and presented during the final presentation period must satisfy all final technical requirements listed in this specification.

Presentation (20 points total)

The final three scheduled course meetings are reserved for oral project presentations. Each individual or team will demonstrate and explain the completed application.

- **Presentation Content and Slides (10 points)**
  - Prepare a presentation of at least **10 slides** that clearly communicates the project and its technical implementation.
  - Recommended sections include:
    - Project title and team members
    - Problem statement
    - Motivation and intended users
    - Major features and functionality
    - Backend architecture and technology stack
    - API integrations
    - Database design and CRUD operations
    - Authentication and RBAC
    - Real-time functionality
    - Deployment workflow and public URL

    - Development challenges and lessons learned
    - Live demonstration or screenshots
  - The **live demo is the most important part** of the presentation. Teams should demonstrate the deployed application whenever possible.

- **Delivery, Demonstration, and Technical Q&A (10 points)**
  - Each presentation is approximately **30 minutes total**: about 20 minutes for presentation/demo and 10 minutes for technical Q&A.
  - All team members must participate.
  - Students must demonstrate that the deployed backend functions with the required features.
  - Students should be prepared to answer technical questions about architecture, routes, database logic, APIs, authentication, authorization, deployment, and code organization.
  - Team members should be able to explain their own contributions and demonstrate familiarity with the overall system.

Extra Credit Opportunities (maximum 10 points)

Students may earn up to **10 extra-credit points** by implementing one or more advanced features that meaningfully extend the project beyond the core requirements. Completing multiple advanced features does not guarantee more than 10 points unless otherwise announced.

Design and User Experience

- **Polished Frontend Integration**
  - Build a user-friendly and visually polished frontend that clearly demonstrates the backend functionality.
  - Consider accessibility, layout consistency, responsive design, readable typography, and professional presentation.
  - Frontend quality is not a core focus of the course, but a strong interface can significantly improve the usability and presentation of the project.

Advanced Functionality

- **Generative AI / LLM Integration**
  - Integrate an LLM API or service such as OpenAI GPT, Google Gemini, Anthropic Claude, DeepSeek, Llama-based services, or another instructor-approved platform.
  - Possible features include summarization, question answering, recommendation, conversational assistance, planning, or generated content.
  - The AI feature must be integrated into the application workflow rather than used only during development.

- **Streaming and Multi-Modal Content**
  - Implement backend functionality for handling real-time video, audio, webcam, speech, or other streaming/multi-modal content.

  - Examples include live audio/video streaming, speech processing, or server-side media transformation.

- **Advanced Multi-User Interaction**
  - Implement substantial multi-user synchronization beyond the minimum real-time requirement.
  - Examples include shared documents, collaborative boards, video calling, group chat, live-stream interactions, multiplayer game state, or collaborative task management.

Scalability and Performance

- **Load Testing and Optimization**
  - Conduct load testing and analyze performance under higher traffic.
  - Optimize database queries, API calls, or server responses based on measured results.
- **Horizontal Scaling or Auto-Scaling**
  - Deploy multiple application instances or containers and use a load balancer or cloud auto-scaling feature.
  - Docker, Kubernetes, AWS, Azure, Google Cloud, or another suitable platform may be used.
- **Caching**
  - Implement Redis, Memcached, CDN caching, or another appropriate caching mechanism to reduce repeated computation or database load.
- **Database Sharding or Advanced Data Distribution**
  - Partition or distribute data using an appropriate sharding or scaling strategy and explain why the design is useful.
- **Search Engine Integration**
  - Integrate a search platform such as Elasticsearch, OpenSearch, Algolia, or another appropriate technology to provide advanced search over substantial project data.

Submission Checklist

Each individual or team must submit a complete and functional project through Canvas. Use the checklist below to verify that all required materials and technical requirements are complete before submission.

☐ GitHub repository URL

☐ Complete backend source code

☐ Complete `README.md`

☐ Local setup and run instructions

☐ Required environment-variable documentation

☐ Public deployment URL

☐ Identification and documentation of all required APIs

☐ Database setup and data-model documentation

☐ Authentication and RBAC documentation

☐ Presentation slides

☐ Live demonstration ready

☐ At least **3 integrated APIs**

☐ At least **1 public API**

☐ At least **1 private/protected API**

☐ Complete database-backed **CRUD operations**

☐ Real-time or automatically synchronized data functionality

☐ User authentication

☐ At least **2 user roles** with meaningful RBAC

☐ Protected routes or endpoints

☐ Publicly accessible deployed backend

☐ Meaningful Git history and team contributions

The project must build, run, and remain accessible for grading. If the application requires API credentials, configuration files, backend services, private keys, or other setup information that cannot be committed publicly, students must provide clear instructions for configuring and running the project.

For team projects, Git history, project check-ins, code contributions, oral questioning, and the final presentation may be used to evaluate individual contributions.

Example Project Ideas

Project Idea	**Possible Core Features**	**Possible APIs / Backend Components**
Personal Finance Tracker	Expense tracking, budget management, transaction history, savings goals, alerts, and financial insights.	Currency or finance API; private user-data API; database for transactions and budgets; RBAC for user/admin functions.
Weather-Based Activity Planner	Weather-aware activity suggestions, saved locations, user preferences, planning, and notifications.	Weather API; location/maps API; private preference API; database for locations and plans.
Recipe Finder with AI Suggestions	Recipe search, meal plans, dietary preferences, favorites, shopping lists, and AI-assisted recommendations.	Recipe/nutrition API; LLM or recommendation API; private user-data API; database for favorites and plans.
Real-Time Chat Application	User accounts, direct/group messaging, presence indicators, message history, moderation, and optional AI assistance.	WebSockets or Socket.IO; sentiment/moderation API; private chat API; database for users and messages.
Task Management System	Create and assign tasks, deadlines, priorities, team collaboration, notifications, and progress tracking.	Calendar API; private task API; real-time updates; RBAC for manager/member roles; database for tasks and teams.

Project Idea	Possible Core Features	Possible APIs / Backend Components
Travel Itinerary Builder	Trip planning, destinations, saved places, schedules, recommendations, shared itineraries, and notes.	Maps/location API; weather API; travel or LLM API; database for itineraries and user preferences.
Turn-Based or Card Game	Multiplayer accounts, turn-based gameplay, persistent game state, rankings, match history, and leaderboards.	Card/randomization API if appropriate; private game API; WebSockets; database for game state, users, and scores.
Restaurant Finder	Restaurant search, filters, saved favorites, reviews, reservations, location-aware recommendations, and user preferences.	Yelp/Google Places or similar API; maps API; private user API; database for favorites, reviews, and reservations.
Matchmaking Application	User profiles, preferences, matching logic, messaging, privacy controls, and moderation.	Location or demographic API where appropriate; private profile/matching API; WebSockets; database for users and matches.
Club or Organization Manager	Membership management, events, announcements, attendance, roles, voting, and shared resources.	Calendar API; private member API; RBAC for officers/members; real-time announcements; database for members and events.
Event Tracking Application	Create and discover events, follow events, receive notifications, manage attendance, and personalize recommendations.	Ticketmaster/Eventbrite or similar API; notification service; private preference API; database for events and users.
Movie / TV Application	Search titles, browse details, ratings, watchlists, reviews, recommendations, and viewing history.	TMDb or similar media API; recommendation API; private user API; database for watchlists and reviews.
Real-Time Translation Application	Translate text or speech, save translation history, user preferences, live conversation support, and multilingual interaction.	Translation API; speech-to-text API; text-to-speech API; private user API; database for preferences/history.
Utility Management System	Usage tracking, billing, payment reminders, account history, service requests, analytics, and notifications.	Utility-rate API; payment API such as Stripe; private customer API; database for usage, bills, and accounts.
Music Discovery / Playlist Application	Search artists, albums, and tracks; create playlists; save favorites; manage user profiles; generate recommendations; and support shared or collaborative playlists.	Spotify Web API, Last.fm API, MusicBrainz API, Discogs API; private API for users and playlists; database for favorites, history, ratings, and preferences.

Note: These examples are intended only as starting points. Students are encouraged to propose original project ideas that satisfy the technical requirements and are appropriately scoped for the semester.